

SALES AI AGENT PLATFORM

Knowledge-Based Agent

System Architecture & Technical Design

The core knowledge and context layer of a centralized six-agent Sales AI automation system

Document	Version	Primary Focus	System Context
Knowledge-Based Agent: Technical Design	1.1 (revised)	Knowledge and context intelligence	Centralized Sales AI Platform

Prepared using the supplied Oldy Buddy System Architecture & MVP Flowchart documents as architectural reference.

1. Purpose and Project Context

This document defines the architecture and technical design of the Knowledge-Based Agent. It is the shared intelligence layer that collects, organizes, retrieves, validates and assembles the information the other sales agents need. The design follows one lifecycle: ingest → understand → store → retrieve → assemble context → ground agent decisions → update.

Overall project context: The Sales AI Agent Platform consists of six specialized agents: Knowledge-Based, Lead Research, Campaign Content, Outreach, Inbox/Reply Intelligence, and Lead Lifecycle/Next Action. The Knowledge-Based Agent acts as the centralized knowledge and context layer, providing trusted, relevant and up-to-date information to the other agents.

Core principle: The Knowledge-Based Agent is not a generic chatbot or a single vector database. It is a governed knowledge and context service that combines structured business data, semantic knowledge, event history and current interaction context.

2. Problem Statement and Goals

The problem

- Sales knowledge is scattered across CRM records, documents, playbooks, emails, calls and notes.
- Different agents may work from different or incomplete versions of the same lead or company.
- Vector search alone cannot provide reliable business state, permissions, recency or lifecycle context.
- LLMs can produce plausible but unsupported statements if evidence is not retrieved before they reason.
- Every interaction changes a lead's state, so knowledge must be updated from events, not kept as static documents.

What the agent must achieve

- **One trusted layer:** give all six agents the same relevant, current and permission-aware context.
- **Combined knowledge:** unite structured data, semantic knowledge and event history, and keep the relationships between leads, people, companies, campaigns, products and conversations.
- **Task-based retrieval:** return evidence for the current task, not undifferentiated information.
- **Trust and provenance:** keep source, timestamp and confidence for key knowledge, and stop unsupported claims reaching customers.
- **Continuous updates:** learn from new research, outreach, replies, meetings, CRM changes and human corrections.

- **Clean interface:** expose an API and event interface so other agents never need to know how knowledge is stored.

3. Role in the Overall Platform

The Knowledge-Based Agent is a shared dependency, not a step in a linear chain. Agents request knowledge from it, and their completed actions produce new events that are written back into the knowledge layer.

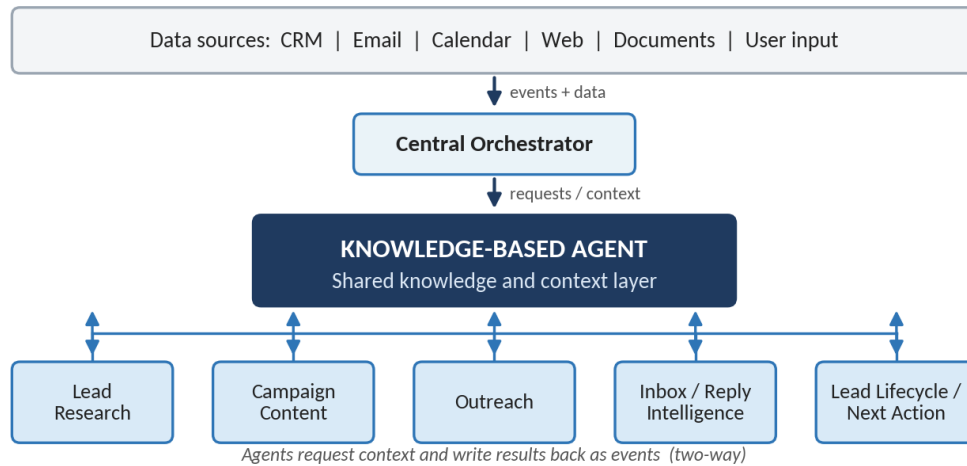


Figure 1: Position of the Knowledge-Based Agent in the Sales AI platform

Functional responsibilities

Capability	Responsibility
Ingestion and normalization	Brings in documents, CRM data, communications, research and events, and converts them into consistent entities, fields, metadata and timestamps.
Entity resolution	Links people, leads, companies, campaigns, products and conversations correctly.
Knowledge storage	Maintains structured records, semantic representations and historical events.
Retrieval and context assembly	Finds information relevant to a specific agent task and packages it as compact, task-specific context.
Grounding and provenance	Exposes sources and evidence, and records where knowledge came from, when it was observed and how it was transformed.
Updates and conflict handling	Applies new facts, research and human corrections; detects conflicting information and applies source and recency rules.
Access control	Ensures retrieved knowledge respects user, tenant and agent permissions.
Context freshness	Refreshes stale information before high-impact actions.

4. Knowledge-Based Agent Architecture

Figure 2 shows the internal architecture. Knowledge flows downward through six layers: sources, ingestion, processing, storage, retrieval and context assembly. The result, a ContextPackage, goes to the orchestrator and agents. Their results return as events (green arrow), and governance applies to every layer.

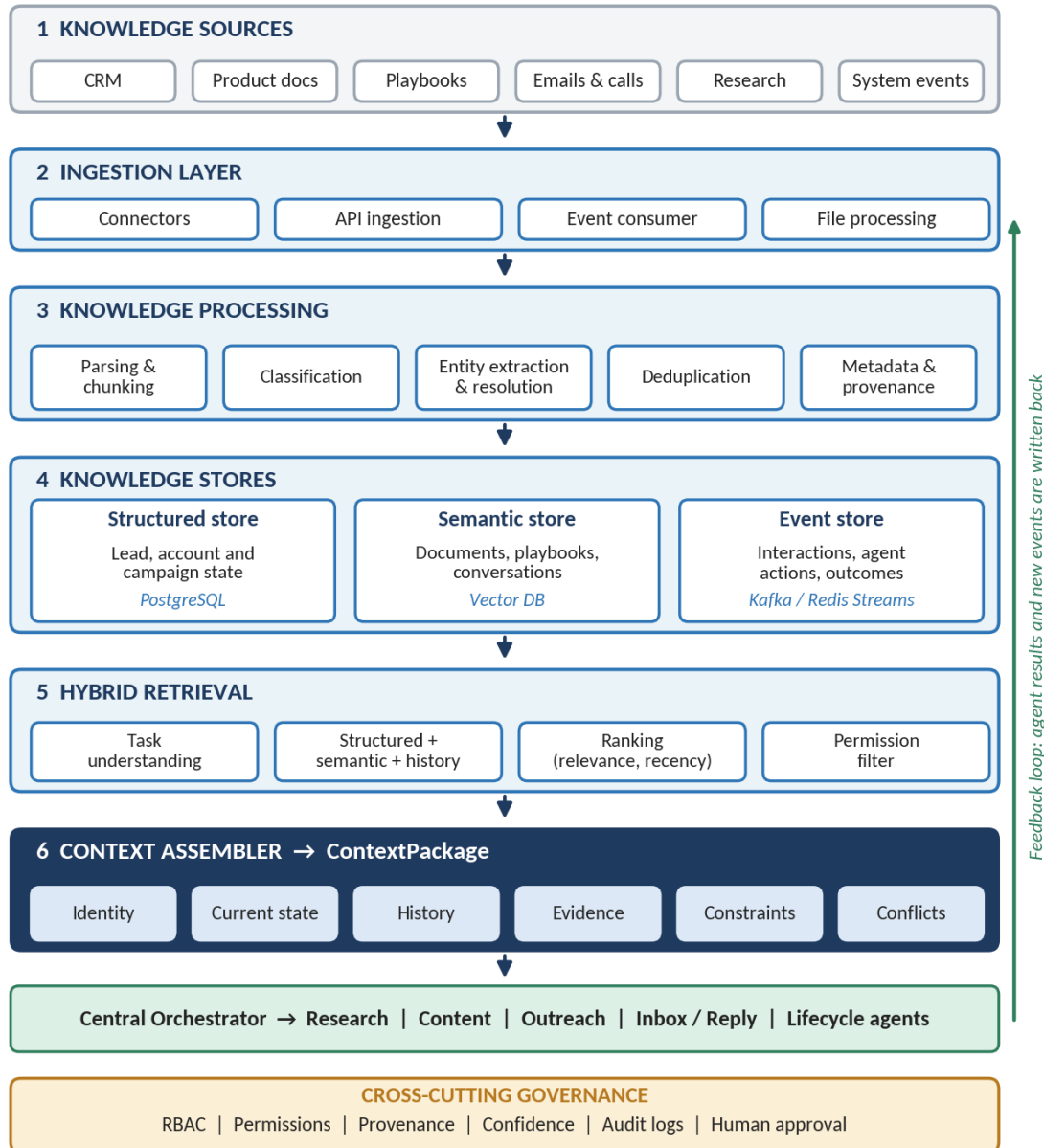


Figure 2: Knowledge-Based Agent architecture

5. Knowledge Domains

The agent organizes sales knowledge into nine domains, each serving a different purpose.

Domain	Examples	Why it matters
Company knowledge	Products, services, positioning, case studies, pricing rules	Grounds customer-facing claims
Customer / account	Company profile, industry, size, technology, business signals	Supports research and personalization

Domain	Examples	Why it matters
Contact / lead	Role, persona, qualification, preferences	Determines relevance and messaging
Conversation	Emails, replies, calls, meeting notes	Maintains continuity and intent
Campaign	Objective, audience, messaging, sequence, content	Keeps campaigns consistent
Sales playbooks	ICP, qualification, objection handling, lifecycle rules	Provides governed sales guidance
Activity history	Outreach, engagement, meetings, CRM changes	Records what actually happened
Research knowledge	News, hiring, expansion, strategic signals	Adds current external evidence
User corrections	Seller edits, approvals, overrides	Improves trust and future context

6. Knowledge Lifecycle and Updates

Knowledge is never static. Every agent action creates new information, which flows back into the same lifecycle.

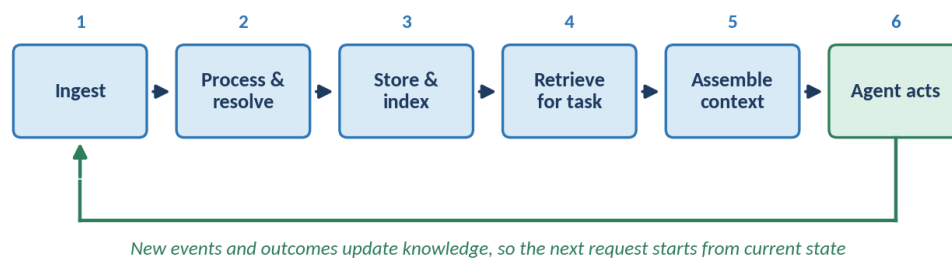


Figure 3: Knowledge lifecycle

Ingestion and processing pipeline

1. **Receive** source data from the CRM, email, document stores, research tools, user input or platform events.
2. **Parse and chunk** each source into usable text and structured fields (for example, text from a PDF, email or transcript).
3. **Normalize and classify** names, dates and identifiers; identify document type, knowledge domain and sensitivity.
4. **Extract entities and facts** such as person, company, product, campaign, pain point and interaction; store key facts separately from document text.
5. **Resolve and deduplicate** against existing records so 'ABC Tech' is linked to the existing ABC Technologies account instead of creating a copy.
6. **Attach metadata:** source, timestamp, author or system, confidence, permissions and version.
7. **Persist and announce:** write to the structured, semantic and event stores, then publish a knowledge-updated event when material information changes.

Short workflow: new lead to personalized outreach

1. A new lead enters the CRM and the orchestrator requests its context from the Knowledge-Based Agent.
2. The agent resolves the person and company, then retrieves existing knowledge, history and campaign data.
3. The Research Agent researches only what is missing; findings return with sources and timestamps, and an updated ContextPackage goes to the Campaign Content and Outreach agents.
4. The outreach event is recorded, and later replies update the knowledge layer and trigger the next lifecycle decision.

Event-driven updates

Knowledge stays current through events, each triggering a specific update:

Event	Knowledge action
LEAD_CREATED	Create or resolve the lead and account entities.
RESEARCH_COMPLETED	Store findings, evidence and timestamp.
OUTREACH_SENT	Record the communication and update activity state.
EMAIL_RECEIVED / REPLY_CLASSIFIED	Store the message, link it to the conversation and save intent or objection signals.
MEETING_BOOKED / COMPLETED	Update lifecycle state; store the outcome and notes.
CRM_UPDATED	Synchronize authoritative structured state.
HUMAN_CORRECTION	Record the corrected fact with an audit trail.
CAMPAIGN_UPDATED	Refresh campaign context and invalidate affected cached context.

7. Knowledge Representation and Data Stores

No single store suits every kind of sales knowledge, so the agent keeps three representations: structured data (state and relationships), semantic knowledge (documents, playbooks, conversations) and event history (what happened and when). Retrieval combines all three.

Store	Purpose	Example technology
Relational database	System of record for structured sales state	PostgreSQL
Vector database	Semantic retrieval of documents and conversations	Production vector DB with embeddings
Event store / broker	Chronological events and asynchronous agent triggers	Kafka, Redis Streams or equivalent
Cache	Short-lived context, jobs, rate limits and workflow state	Redis
Object storage	Original documents, transcripts and artifacts	S3-compatible storage
LLM / embedding service	Reasoning, extraction, classification and embeddings	Enterprise LLM and embedding provider

8. Retrieval and Context Assembly

Retrieval is hybrid. Semantic similarity finds relevant text, but sales decisions also depend on exact identifiers, lifecycle state, permissions and recency. Each request follows the same path: the agent request is analyzed, a retrieval plan queries the structured, semantic and event stores in parallel, results are ranked by relevance, recency, source quality and confidence, restricted items are removed by the permission filter, and the remainder is assembled into context.

The output is a structured ContextPackage, not a raw list of search results:

Context section	Typical contents
Identity	Lead ID, contact, company, role, account relationships
Current state	Lifecycle stage, campaign, owner, open tasks, pending action
Relevant knowledge	Product and service information, playbook guidance, relevant documents
Interaction history	Recent and important communications and meetings
Research and signals	Current research findings, intent, engagement and qualification signals
Constraints	Messaging and channel rules, permissions, approval requirements
Evidence	Sources, timestamps, confidence and provenance
Conflicts	Known discrepancies that require caution or resolution

Context freshness. Stable, frequently used knowledge can be cached, while current lead state and recent interactions use short freshness windows. High-impact actions trigger a refresh when material information may have changed; updates invalidate affected cached context, and cached context always carries a timestamp and version.

9. Knowledge Quality, Provenance and Confidence

- Every important fact keeps a source reference (external research also keeps its URL and observation time), and observed facts are kept separate from AI-generated inferences.
- Stale information is marked or deprioritized, never silently treated as current.
- Conflicting sources are preserved as conflicts until defined source and recency rules resolve them.
- Human corrections are auditable and never overwrite history without a recorded change; downstream agents receive enough provenance to avoid presenting weak evidence as fact.

Knowledge type	Example	Recommended treatment
Verified structured fact	CRM account owner	High confidence; direct use
Approved company knowledge	Official product capability	High confidence; preferred source
Recent external evidence	Company hiring signal	Use with source and timestamp
Conversation-derived fact	Prospect says budget is approved	Attribute to the conversation; keep timestamp
AI inference	Likely pain point	Label as inference; never present as fact
Conflicting or stale information	Two company sizes; an old job title	Flag the conflict and prefer the governed source; refresh stale data before use

10. Agent Interface and How Other Agents Use Knowledge

Endpoint	Purpose
GET /knowledge/context/{lead_id}	Return the assembled ContextPackage for a lead
POST /knowledge/retrieve	Run a task-specific retrieval
POST /knowledge/ingest /update	Submit source data; apply new facts or events
GET /knowledge/entity/{id} /history/{id}	Fetch an entity record and its change history
POST /knowledge/feedback /resolve-conflict	Record feedback; resolve a flagged conflict

Example ContextPackage

```
{ "lead": {...}, "company": {...}, "current_state": {...},  
  "relevant_knowledge": [...], "interaction_history": [...],  
  "research_signals": [...], "constraints": [...],  
  "evidence": [...], "conflicts": [...],  
  "context_timestamp": "...", "retrieval_confidence": 0.0 }
```

How the other five agents consume knowledge

Agent	Knowledge request	Knowledge response
Lead Research	What is already known, and what is missing?	Existing profile, gaps, prior research and source history
Campaign Content	What is relevant for this message?	Persona, pain points, product evidence, approved messaging, recent interactions
Outreach	What must be known before execution?	Lead and campaign state, previous sends, channel constraints, timing
Inbox / Reply Intelligence	What context is needed to interpret this reply?	Conversation history, lead profile, campaign, previous objections, relevant knowledge
Lead Lifecycle	What is the current complete state?	Lifecycle, interactions, engagement, research, playbook rules, open tasks

11. Security, Governance and Failure Handling

- Role-based access control for users and agents, with agent-specific tool and data permissions.
- Tenant and account isolation where required; source-level permissions carried into retrieval results.
- Encryption in transit and at rest, plus audit logs for knowledge access, changes and agent decisions.
- Protection against prompt injection hidden in external documents, websites or emails.
- Sensitive-information filtering before data reaches downstream agents, and human approval for corrections or actions with material commercial impact.

Failure	Knowledge-Based Agent response
Source unavailable	Return the available knowledge and explicitly mark the source as unavailable.
Retrieval or vector failure	Fail safely and never invent missing context; fall back to structured or keyword retrieval where possible.
Entity ambiguity	Request resolution or keep a safe unresolved state.
Conflicting or stale knowledge	Return conflict metadata instead of silently choosing a source; mark stale items and refresh when needed.
LLM extraction failure	Retry or use deterministic parsing; preserve the raw source.
Permission failure	Exclude restricted knowledge and record the access decision.
Duplicate event	Use idempotency and event IDs to prevent duplicate updates.

12. Evaluation, Implementation Stack and MVP Scope

Evaluation and quality metrics

Metric	What it measures
Retrieval precision and recall	Whether retrieved items are relevant and whether important knowledge is found.
Groundedness	Whether agent outputs are supported by retrieved evidence.
Freshness	Whether current facts are preferred over stale ones.
Entity resolution accuracy	Whether information is linked to the correct lead or company.
Provenance coverage and conflict detection	Share of important facts with traceable sources, and ability to surface contradictions.
Context usefulness	Whether downstream agents can complete their tasks with the ContextPackage.
Latency and cost	Time and spend per context request (retrieval, embedding and LLM).

Recommended implementation stack

This is a direction, not a fixed choice; vendors can be finalized during engineering.

Layer	Recommended direction
Backend API	Python + FastAPI
Data and cache	PostgreSQL for structured data; Redis + workers for cache and jobs; object storage for documents
Semantic retrieval and AI	Production vector database; enterprise LLM and embedding models
Orchestration	Application-level orchestrator with event-driven workflows
Frontend and auth	React/TypeScript or existing sales application; OAuth2/OIDC + JWT + RBAC
Deployment	Docker-based cloud deployment

MVP scope

The MVP should prove the core value without trying to build a full enterprise knowledge graph on day one:

- CRM lead and company synchronization; product, playbook and email/conversation ingestion.
- Basic entity resolution and hybrid structured + semantic retrieval.
- Lead-level ContextPackage generation with source and provenance tracking.
- Knowledge update events, basic freshness and conflict handling.
- APIs for the other five agents, with audit logging and RBAC.

13. Conclusion

Design principle: Know what the system knows → determine what is relevant → retrieve evidence → assemble current context → expose provenance and uncertainty → support agent reasoning → record the outcome → update knowledge.

The Knowledge-Based Agent is the foundation of the centralized Sales AI architecture. The other five agents should not keep their own versions of company, lead, campaign and conversation knowledge; they request task-specific context from this shared layer and return new information through structured updates and events.